

WEEK-5 TASK REPORT

Payment Transaction Management System

Student Name:	Harineeshwaran.V
Enrollment Number:	TU6253210211020
Task Module:	Payment Transaction Management & Reporting
Core Framework:	Relational Database Management System (DBMS – SQL)

1. Task Design & Architectural Overview

Modern e-commerce and banking platforms process a high volume of customer payments across multiple modes such as UPI, credit card, debit card, and net banking. This module designs a Payment table within the relational schema to record every transaction attempt, capture its outcome, and support downstream reporting. The design focuses on maintaining transactional integrity between successful and failed payments while enabling fast retrieval for analytics.

2. Objectives

- Create a Payment table linked to the Orders and Customers tables.
- Store payment details such as mode, date, and status for every transaction.
- Manage and distinguish successful and failed transactions reliably.
- Analyze payment methods most frequently used by customers.
- Generate consolidated payment transaction reports for business review.

3. Database Schema & Implementation

The Payment table stores one row per transaction attempt, referencing the related order and customer. A CHECK constraint restricts the status column to valid states, and an index on payment_date supports efficient reporting queries.

```
CREATE TABLE Payment (  
    payment_id      INT PRIMARY KEY AUTO_INCREMENT,  
    order_id        INT NOT NULL,  
    customer_id     INT NOT NULL,  
    payment_mode    VARCHAR(20) NOT NULL,    -- UPI, Card, NetBanking, Wallet  
    payment_date    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    amount          DECIMAL(10,2) NOT NULL,  
    status          VARCHAR(10) NOT NULL,    -- Success, Failed, Pending  
    CONSTRAINT chk_status CHECK (status IN ('Success', 'Failed', 'Pending')),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);  
  
CREATE INDEX idx_payment_date ON Payment(payment_date);
```

4. Transaction Management Queries

Insert a new payment record:

```
INSERT INTO Payment (order_id, customer_id, payment_mode, amount, status)  
VALUES (1042, 205, 'UPI', 2499.00, 'Success');
```

Retrieve all failed transactions for follow-up:

```
SELECT payment_id, order_id, customer_id, payment_mode, payment_date, amount
FROM Payment
WHERE status = 'Failed'
ORDER BY payment_date DESC;
```

Update a pending transaction once confirmed by the gateway:

```
UPDATE Payment
SET status = 'Success'
WHERE payment_id = 5031 AND status = 'Pending';
```

5. Payment Method Analysis

The query below groups completed transactions by payment mode to identify the most popular methods among customers, which supports gateway cost optimization and UX prioritization.

```
SELECT payment_mode,
       COUNT(*)      AS total_transactions,
       SUM(amount)    AS total_amount,
       ROUND(AVG(amount),2) AS avg_transaction_value
FROM Payment
WHERE status = 'Success'
GROUP BY payment_mode
ORDER BY total_transactions DESC;
```

6. Transaction Report Generation

A daily summary report reconciles successful revenue against failed attempts, giving finance teams visibility into gateway performance and potential customer drop-off points.

```
SELECT DATE(payment_date) AS txn_date,
       status,
       COUNT(*) AS txn_count,
       SUM(amount) AS total_amount
FROM Payment
GROUP BY DATE(payment_date), status
ORDER BY txn_date DESC, status;
```

7. Simulation Results & Observations

Data Integrity: Foreign key and CHECK constraints prevented invalid order references and out-of-range status values during test inserts.

Transaction Segregation: Successful and failed transactions were cleanly separable using the status column, simplifying reconciliation queries.

Reporting Efficiency: Indexing payment_date reduced query response time for date-range reports on the simulated dataset.

8. Conclusion

This component successfully fulfils the Week-5 task requirements, demonstrating a robust Payment Transaction Management System capable of storing transaction details, managing successful and failed payments, analyzing customer payment preferences, and generating actionable transaction reports within a relational database.